

JavaScript

Aula 2: decisões e repetição

O que vamos ver hoje

01 if e o bloco de decisão

02 if / else if / else

03 Operador ternário

04 switch com case e break

05 while e do...while

06 for, break e continue

07 Recursão e caso base

08 Armadilhas e quiz final



01 if: a primeira decisão

O if executa um bloco de código só quando a condição for verdadeira.

decisao.js

```
let nota = 7;

if (nota >= 6) {
  console.log("Aprovado!");
}

console.log("Fim da checagem");

// Aprovado!
// Fim da checagem
```

A condição vira boolean

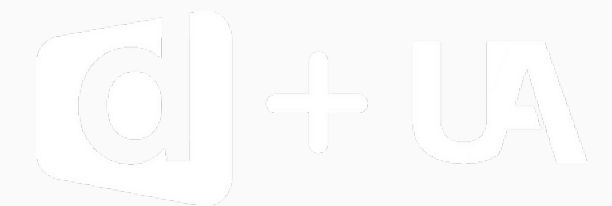
Dentro dos parênteses vai qualquer coisa que resulte em true ou false.

As chaves marcam o bloco

Tudo entre { } roda junto quando a condição é verdadeira.

Sem else, nada acontece

Se a condição for false o if é ignorado e o programa segue em frente.



01

if / else if / else

Quando existem caminhos alternativos, o else assume tudo o que sobrou.

boletim.js

```
let nota = 7;

if (nota >= 9) {
  console.log("Excelente");
} else if (nota >= 6) {
  console.log("Aprovado");
} else if (nota >= 4) {
  console.log("Recuperacao");
} else {
  console.log("Reprovado");
}
// Aprovado
```

A ordem importa

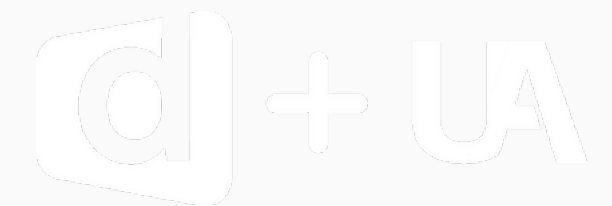
Testa de cima para baixo e para no primeiro true. Vá do mais restrito ao mais amplo.

else não tem condição

É o caminho padrão: roda quando nenhuma condição anterior deu true.

Só um bloco roda

Em um encadeamento if / else if / else, exatamente um caminho é executado.



01 if: erros comuns

Três erros que quebram o seu if — ou pior: não quebram e dão resultado errado.

não faça isso

```
if (nota = 6) { } // atribui, sempre true
if (nota == "6") { } // converte o tipo

if (nota >= 6); // ; encerra aqui
  console.log("ok"); // roda sempre

// jeito certo:
if (nota >= 6) {
  console.log("ok");
}
```

= no lugar de ===

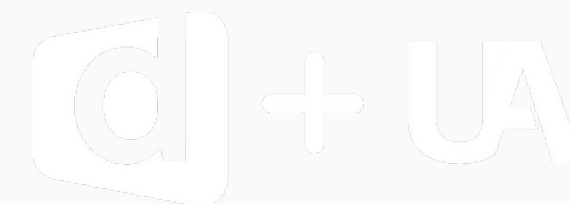
Um sinal só atribui e o if fica sempre verdadeiro. Não dá erro, só resultado errado.

Ponto e vírgula depois do if

if (x > 1); encerra o comando ali. O bloco de baixo passa a rodar sempre.

Sempre use chaves

Mesmo com uma linha só. Evita bug no dia em que você adicionar a segunda.





Boletim do aluno

Classifique uma nota: 9+ Excelente, 6+ Aprovado, 4+ Recuperação, abaixo disso Reprovado.

- use if / else if / else
- teste com 10, 7, 5 e 2
- cuidado com a ordem das condições
- compare sempre com `>=` e `===`

saída esperada no console

```
nota = 10    ->  Excelente
nota = 7     ->  Aprovado
nota = 5     ->  Recuperacao
nota = 2     ->  Reprovado
```



02

Operador ternário

Um if/else de uma linha que, em vez de executar um bloco, devolve um valor.

ternario.js

```
// condicao ? seTrue : seFalse

let nota = 7;
let status = nota >= 6
  ? "Aprovado"
  : "Reprovado";

console.log(status);    // Aprovado

// o mesmo que 5 linhas de if/else
```

Ele devolve um valor

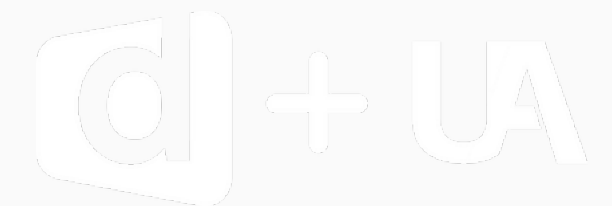
Por isso cabe numa atribuição, dentro de template string ou de console.log.

? é o então, : é o senão

Leia: se a condição for verdadeira use o primeiro valor, senão use o segundo.

Não aninhe demais

Dois ternários encaixados já ficam ilegíveis. Aí volte para o if / else.



Entrada no evento

Use um ternário para decidir a mensagem de acesso a partir da idade da pessoa.

- condição: idade $\geq$ 18
- guarde o resultado em uma variável
- imprima com template string
- depois reescreva com if/else e compare o tamanho

saída esperada no console

```
idade = 16    ->  Acesso: Negado  
idade = 20    ->  Acesso: Liberado
```


03

switch: menu de opções

Use quando você compara a MESMA variável com vários valores fixos.

agenda.js

```
let dia = 3;

switch (dia) {
  case 1:
    console.log("Segunda");
    break;
  case 3:
    console.log("Quarta");
    break;
  default:
    console.log("Outro dia");
}
// Quarta
```

Compara com ===

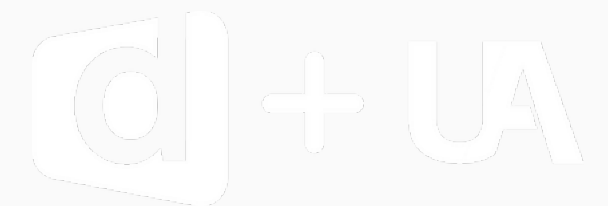
O switch usa comparação estrita: o case "3" não pega o número 3.

break encerra o case

Sem break a execução escorre para o case seguinte. Isso se chama fall-through.

default é o else do switch

Roda quando nenhum case combina. Coloque sempre, mesmo que só para avisar.



03

switch: onde ele falha

O switch é ótimo para menus e péssimo para faixas de valor.

cuidado.js

```
switch (nota) {  
  case nota >= 6:    // NAO funciona  
    console.log("Aprovado");  
}  
  
// para faixa (>=, <) use if / else if  
  
switch (opcao) {  
  case 1:  
  case 2:            // agrupando cases  
    console.log("1 ou 2");  
    break;  
}
```

Não serve para faixas

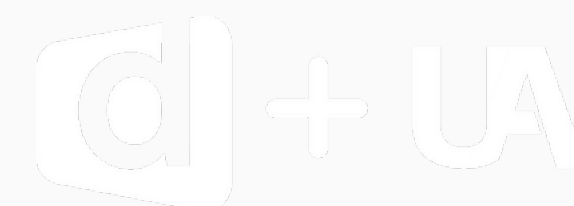
case compara igualdade. Para $\geq$ e $\leq$ o caminho certo é if / else if.

Esquecer o break

A execução continua no case de baixo e você vê duas mensagens.

Cases agrupados

Dois case seguidos, sem código entre eles, tratam os dois valores igual.



Cantina da faculdade

Monte o menu: 1 café, 2 pão de queijo, 3 suco. Qualquer outro número mostra Opção inválida.

- use switch com o número da opção
- não esqueça o break em cada case
- use default para a opção inválida
- teste com 2 e depois com 9

saída esperada no console

```
opcao = 1    ->  Cafe
opcao = 2    ->  Pao de queijo
opcao = 3    ->  Suco
opcao = 9    ->  Opcao invalida
```

04 while: repetir enquanto

Repete enquanto a condição continuar verdadeira. O contador é responsabilidade sua.

while.js

```
let i = 1;

while (i <= 3) {
  console.log(`Volta ${i}`);
  i++;           // sem isso: infinito
}

// Volta 1
// Volta 2
// Volta 3
```

Três partes obrigatórias

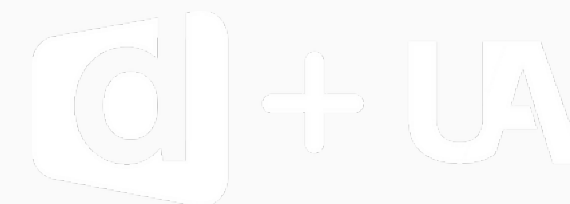
Iniciar o contador antes, testar na condição e atualizar dentro do bloco.

Pode rodar zero vezes

Se a condição já começa false, o bloco nunca é executado.

Use quando não sabe o total

Ex: repetir até o usuário digitar sair. Se você sabe o número de voltas, use for.



04

do...while e o loop infinito

O do...while roda pelo menos uma vez. E o loop infinito é o erro nº 1 do while.

do-while.js

```
let opcao;

do {
  opcao = 2;          // roda sempre 1x
  console.log("mostrando menu");
} while (opcao !== 2);

// ERRO CLASSICO: esqueceu o i++
let i = 0;
while (i < 3) {
  console.log(i);    // 0 para sempre
}
```

do roda antes de testar

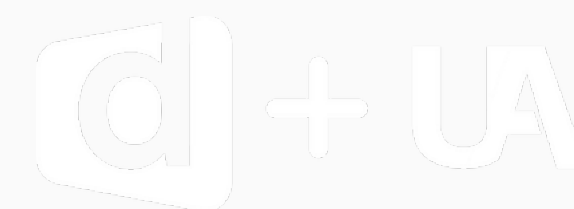
Perfeito para menu: mostra as opções uma vez e repete se a escolha for inválida.

Loop infinito trava a aba

Se a condição nunca fica false o navegador congela. Feche a aba para sair.

Checklist do while

Contador criado antes? A condição pode virar false?
Atualiza dentro do bloco?



05

for: repetir contando

Quando você já sabe quantas voltas vai dar, o for junta as três partes em uma linha.

for.js

```
// for (inicio; condicao; passo)

for (let i = 1; i <= 3; i++) {
  console.log(`Volta ${i}`);
}

let materias = ["Mat", "Port", "Hist"];

for (let i = 0; i < materias.length; i++) {
  console.log(i, materias[i]);
}
```

As três partes juntas

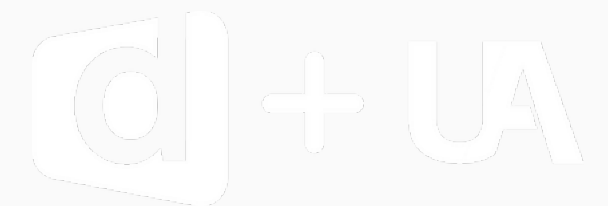
Inicialização, condição de parada e passo, separados por ponto e vírgula.

Percorrendo array

Comece em 0 e vá até length - 1, ou seja: `i < materias.length`.

for...of é mais limpo

Quando você não precisa do índice, use `for (let m of materias)`.



05

break e continue

Duas palavras que mudam o rumo de qualquer loop no meio do caminho.

controle.js

```
for (let i = 1; i <= 10; i++) {  
  if (i === 4) break;      // sai do loop  
  console.log(i);          // 1 2 3  
}
```

```
for (let i = 1; i <= 5; i++) {  
  if (i % 2 === 0) continue; // pula  
  console.log(i);            // 1 3 5  
}
```

break sai do loop

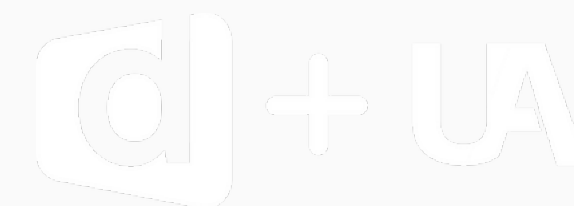
Encerra a repetição na hora. Ótimo para parar quando já achou o que procurava.

continue pula só a volta

Ignora o resto do bloco e vai para a próxima iteração. O loop continua.

Não abuse

Três break no mesmo loop já é sinal de que a condição podia ser melhor.





Tabuada e contagem

Imprima a tabuada do 7 usando for e faça uma contagem regressiva de 5 até 1 usando while.

- for: i de 1 até 10, imprima $7 * i$
- use template string na saída
- while: comece em 5 e vá decrementando
- extra: pule o resultado de 7×5 com continue

saída esperada no console

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70

5 4 3 2 1
```

Desafio extra

Imprima a tabuada do 1 ao 10 usando dois for encaixados, com uma linha em branco entre elas.



06

Recursão: a função que se chama

Uma função que chama ela mesma, sempre com um caso base para poder parar.

recursao.js

```
function contar(n) {  
  if (n === 0) {           // caso base  
    console.log("Fim!");  
    return;  
  }  
  console.log(n);  
  contar(n - 1);           // chama de novo  
}  
  
contar(3);  
// 3   2   1   Fim!
```

Sempre um caso base

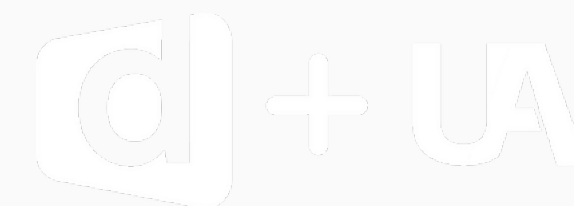
É a condição que faz a função parar de se chamar. Sem ele o programa quebra.

Cada chamada é nova

n vale 3, depois 2, depois 1. Cada chamada tem o seu próprio n.

Pense de trás para frente

Resolva o caso mais simples primeiro e depois reduza o problema até ele.



06

Recursão: armadilhas

Recursão é elegante, mas tem hora e lugar. Na dúvida, um for resolve.

fatorial.js

```
// fatorial: 5! = 5*4*3*2*1
function fatorial(n) {
  if (n <= 1) return 1;    // caso base
  return n * fatorial(n - 1);
}
console.log(fatorial(5)); // 120

// sem caso base:
function loop(n) {
  return loop(n - 1);      // RangeError
}
```

Estouro de pilha

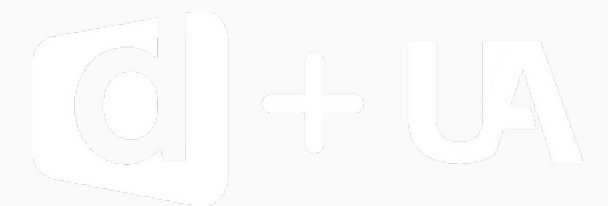
Sem caso base o JavaScript para com "Maximum call stack size exceeded".

Loop é mais barato

Para contar e percorrer array prefira for. Recursão brilha em árvores e pastas.

Todo caminho deve reduzir

Cada chamada precisa se aproximar do caso base, senão a recursão nunca acaba.



Quiz de revisão

Responda sem rodar o código. Depois testamos tudo no console juntos.

1 Qual a diferença entre if / else if e switch?

2 O que acontece se você esquecer o break em um case?

3 Como escrever if (x > 0) ... else ... em um ternário?

4 Quantas vezes roda o bloco de while (false) { ... }?

5 Em for (let i = 0; i < 3; i++), quais valores i assume?

6 O que é o caso base de uma função recursiva?



Bom código!

Toda dúvida é bem-vinda.

